

Android Helios

Rapport Technique du Projet

Application Android : Lampe de poche contrôlée par secousse
Accéléromètre + CameraManager + Graphique en temps réel + Foreground Service

Auteurs : Facundo Arito

Juan Baudino Casais

Matière : Informatique Appliquée à l'Ingénierie

Dépôt : github.com/wopassord/Android_Helios

Contents

1	Introduction	2
2	Structure du Projet	2
3	La Lampe de Poche : CameraManager	3
3.1	API utilisée	3
3.2	Recherche de l'identifiant de la caméra avec flash	3
3.3	Basculement de la lampe de poche	3
3.4	Synchronisation d'état avec TorchCallback	4
3.5	Mise à jour de l'indicateur visuel	4
4	L'Accéléromètre : SensorManager	5
4.1	API utilisée	5
4.2	Cycle de vie : enregistrement et désenregistrement du listener	5
5	Détection de Secousse	6
5.1	Calcul de la magnitude	6
5.2	Algorithme de détection	6
5.3	Constantes et seuil configurable	7
6	Graphique en Temps Réel : AccelerometerGraphView	8
6.1	Vue personnalisée avec Canvas	8
6.2	Tampon circulaire	8
6.3	Initialisation des pinceaux	9
6.4	Méthode onDraw	9
7	Exécution en Arrière-Plan : TorchService	10
7.1	Motivation	10
7.2	Architecture du service	10
7.3	Transfert de contrôle entre Activité et Service	11
7.4	Interrupteur dans l'interface	12
7.5	Déclaration dans le Manifest	12
8	Conclusions	13

1. Introduction

Android Helios est une application Android développée en Java dont l'objectif principal est d'**allumer ou d'éteindre la lampe de poche du téléphone à l'aide d'un geste de secousse configurable**. L'application intègre deux modules matériels :

- **Accéléromètre** : détecte le mouvement du dispositif, calcule la magnitude de l'accélération en unités de pesanteur (g-force) et applique un algorithme de détection de secousse.
- **Caméra / Flash** : contrôle la lampe de poche via `CameraManager.setTorchMode()`, sans avoir besoin d'ouvrir un flux vidéo ni d'accéder physiquement à la caméra.

L'interface propose trois éléments de contrôle : un indicateur d'état de la lampe de poche (cercle coloré + texte), un graphique en temps réel de l'accélération mesurée, et un curseur de sensibilité. Un interrupteur situé en haut à droite permet de maintenir la détection active même lorsque l'utilisateur quitte l'application.

2. Structure du Projet

Le projet suit la structure standard d'Android Studio avec Gradle (Kotlin DSL). Les fichiers développés sont les suivants :

Fichier	Rôle
MainActivity.java	Activité principale. Coordonne l'accéléromètre, la lampe de poche, le graphique en temps réel et toute la logique d'interface.
AccelerometerGraphView.java	Vue personnalisée (<code>extends View</code>) qui trace le graphique de g-force à l'aide du Canvas d'Android.
TorchService.java	Foreground Service. Réplique la détection de secousse et le contrôle de la lampe de poche lorsque l'application est en arrière-plan.
activity_main.xml	Layout principal : indicateur circulaire, graphique, curseur de sensibilité et interrupteur d'arrière-plan.
AndroidManifest.xml	Déclare les permissions (<code>FOREGROUND_SERVICE</code>) et enregistre le service.
drawable/circle_indicator.xml	Shape drawable oval servant d'indicateur visuel de l'état de la lampe de poche.

3. La Lampe de Poche : CameraManager

3.1. API utilisée

Android expose le contrôle du flash via `android.hardware.camera2.CameraManager`. La méthode `setTorchMode(String cameraId, boolean enabled)`, disponible depuis l'API 23, allume ou éteint le flash **sans ouvrir de flux vidéo et sans nécessiter la permission CAMERA**. C'est la raison pour laquelle cette permission n'est pas déclarée dans le manifest : elle est simplement inutile pour notre usage.

3.2. Recherche de l'identifiant de la caméra avec flash

Un appareil peut posséder plusieurs caméras. Avant d'utiliser la lampe de poche, il faut identifier laquelle dispose d'un flash. Cela est résolu dans `findTorchCameraId()`, appelée une seule fois depuis `onCreate()` :

```

1 private String findTorchCameraId() {
2     try {
3         for (String id : cameraManager.getCameraIdList()) {
4             Boolean hasFlash = cameraManager
5                 .getCameraCharacteristics(id)
6                 .get(CameraCharacteristics.
7                     FLASH_INFO_AVAILABLE);
8             if (Boolean.TRUE.equals(hasFlash)) return id;
9         }
10    } catch (CameraAccessException e) {
11        Log.e("MainActivity", "Impossible de trouver la camera
12        torch", e);
13    }
14    return null;
15 }
```

Listing 1: Recherche de l'identifiant de la caméra — MainActivity.java

`getCameraIdList()` retourne un tableau contenant les identifiants de toutes les caméras disponibles. Pour chacune, on consulte la propriété `FLASH_INFO_AVAILABLE` de ses caractéristiques. La méthode retourne le premier identifiant dont le flash est disponible, ou `null` si aucun ne l'est. Si le résultat est `null`, `onShakeDetected()` ne fait rien.

3.3. Basculement de la lampe de poche

Le basculement se produit dans `onShakeDetected()`, déclenchée par l'algorithme de détection lorsque le geste est reconnu :

```

1 private void onShakeDetected() {
2     if (torchCameraId == null) return;
3     try {
4         cameraManager.setTorchMode(torchCameraId, !torchOn);
5     } catch (CameraAccessException e) {
6         Log.e("MainActivity", "Impossible de basculer la torche",
7         e);
8     }
9 }
```

Listing 2: Basculement de la lampe de poche — MainActivity.java

Le champ `torchOn` n'est *pas* modifié ici directement. La source de vérité de l'état réel du flash est le `TorchCallback` décrit ci-après, ce qui évite toute désynchronisation si une autre application ou le `TorchService` modifie le flash en parallèle.

3.4. Synchronisation d'état avec TorchCallback

`CameraManager` fournit un mécanisme de callback pour recevoir des notifications à chaque changement d'état du flash, quelle qu'en soit l'origine :

```

1  private final CameraManager.TorchCallback torchCallback =
2      new CameraManager.TorchCallback() {
3      @Override
4      public void onTorchModeChanged(String cameraId, boolean
5      enabled) {
6          if (cameraId.equals(torchCameraId)) {
7              torchOn = enabled;
8              updateTorchUI();
9          }
10     }
11 };
12 // Dans onCreate() :
13 cameraManager.registerTorchCallback(
14     torchCallback,
15     new Handler(Looper.getMainLooper()));
16
17 // Dans onDestroy() :
18 cameraManager.unregisterTorchCallback(torchCallback);

```

Listing 3: Enregistrement et utilisation du `TorchCallback` — `MainActivity.java`

Le `Handler` avec `Looper.getMainLooper()` garantit que `onTorchModeChanged()` s'exécute sur le thread principal, permettant de mettre à jour l'interface directement depuis le callback.

3.5. Mise à jour de l'indicateur visuel

Lorsque le callback déclenche `updateTorchUI()`, la couleur du `View` circulaire (un `GradientDrawable`) et le texte d'état sont modifiés :

```

1  private void updateTorchUI() {
2      GradientDrawable drawable =
3          (GradientDrawable) viewTorchIndicator.getBackground();
4      if (torchOn) {
5          drawable.setColor(0xFFFFFC107); // ambre
6          tvTorchStatus.setText("ALLUMÉE");
7          tvTorchStatus.setTextColor(0xFFFFFC107);
8      } else {
9          drawable.setColor(0xFF9E9E9E); // gris
10         tvTorchStatus.setText("ÉTEINTE");
11         tvTorchStatus.setTextColor(0xFF9E9E9E);
12     }
13 }

```

Listing 4: Mise à jour de l'indicateur d'état — `MainActivity.java`

La couleur du drawable est modifiée à l'exécution via `setColor()` : le cercle passe du gris (`#9E9E9E`) à l'ambre (`#FFC107`) lorsque la lampe est allumée. C'est la seule fonction qui modifie des éléments d'interface, ce qui centralise toute la logique de présentation en un unique endroit.

4. L'Accéléromètre : *SensorManager*

4.1. API utilisée

Le système de capteurs d'Android est accessible via `android.hardware.SensorManager`. Pour recevoir les lectures de l'accéléromètre, `MainActivity` implémente l'interface `SensorEventListener`, qui impose la définition de deux méthodes : `onSensorChanged()` et `onAccuracyChanged()`.

L'accéléromètre est obtenu et initialisé dans `onCreate()` :

```

1  sensorManager = (SensorManager) getSystemService(Context.
    SENSOR_SERVICE);
2  if (sensorManager != null) {
3      accelerometer = sensorManager.getDefaultSensor(Sensor.
        TYPE_ACCELEROMETER);
4  }
5  if (accelerometer == null) {
6      Log.e("MainActivity", "Cet appareil n'a pas d'accelerometre");
7  }

```

Listing 5: Initialisation de l'accéléromètre — `MainActivity.java`

4.2. Cycle de vie : enregistrement et désenregistrement du listener

Un aspect critique de l'utilisation des capteurs dans Android est la gestion correcte du cycle de vie. Le listener est enregistré uniquement lorsque l'application est au premier plan et désenregistré lorsqu'elle se met en pause, ce qui économise la batterie :

```

1  @Override
2  protected void onResume() {
3      super.onResume();
4      stopService(new Intent(this, TorchService.class));
5      if (accelerometer != null) {
6          sensorManager.registerListener(
7              this,
8              accelerometer,
9              SensorManager.SENSOR_DELAY_NORMAL);
10     }
11 }
12
13 @Override
14 protected void onPause() {
15     super.onPause();
16     if (accelerometer != null) {
17         sensorManager.unregisterListener(this);
18     }
19     if (backgroundEnabled) {

```

```

20         Intent intent = new Intent(this, TorchService.class);
21         intent.putExtra(TorchService.EXTRA_THRESHOLD,
22             shakeThreshold);
23         intent.putExtra(TorchService.EXTRA_TORCH_STATE, torchOn);
24         startForegroundService(intent);
25     }

```

Listing 6: Gestion du cycle de vie du capteur — MainActivity.java

SENSOR_DELAY_NORMAL définit une fréquence d'échantillonnage d'environ 200 ms entre les lectures. Dans `onResume()`, le `TorchService` est également arrêté s'il était en cours d'exécution, afin de garantir qu'à tout moment *un seul* composant écoute le capteur.

5. Détection de Secousse

5.1. Calcul de la magnitude

L'accéléromètre fournit trois valeurs : l'accélération sur les axes X , Y et Z exprimée en m/s^2 . Pour détecter une secousse indépendamment de l'orientation du téléphone, on calcule la **magnitude du vecteur d'accélération** normalisée en unités de pesanteur :

$$g_{\text{force}} = \sqrt{\left(\frac{a_x}{g}\right)^2 + \left(\frac{a_y}{g}\right)^2 + \left(\frac{a_z}{g}\right)^2}$$

où $g = 9,80665 \text{ m/s}^2$ (`SensorManager.GRAVITY_EARTH`). Au repos, le vecteur pointe dans la direction de la pesanteur avec une magnitude $\approx 1 g$. Une secousse produit des pics entre $2,5 g$ et $5 g$.

5.2. Algorithme de détection

La fonction `onSensorChanged()` s'exécute à chaque nouvelle lecture du capteur et concentre toute la logique de détection :

```

1  @Override
2  public void onSensorChanged(SensorEvent event) {
3      float gX = event.values[0] / SensorManager.GRAVITY_EARTH;
4      float gY = event.values[1] / SensorManager.GRAVITY_EARTH;
5      float gZ = event.values[2] / SensorManager.GRAVITY_EARTH;
6      float gForce = (float) Math.sqrt(gX * gX + gY * gY + gZ * gZ);
7
8      graphAccelerometer.addValue(gForce); // mise à jour du
9      graphique
10     if (gForce < shakeThreshold) return; // filtre 1 : seuil
11
12     long now = System.currentTimeMillis();
13     if (now - lastShakeTime < SHAKE_SLOP_TIME_MS) return; //
14     filtre 2
15     if (now - lastShakeTime > SHAKE_COUNT_RESET_TIME_MS) {

```

```

16         shakeCount = 0;                                // filtre 3 : fenetre
           temporelle
17     }
18
19     lastShakeTime = now;
20     shakeCount++;
21
22     if (shakeCount >= MIN_SHAKE_COUNT) {
23         shakeCount = 0;
24         onShakeDetected();
25     }
26 }

```

Listing 7: Détection de secousse en temps réel — MainActivity.java

L'algorithme fonctionne en quatre couches de filtrage superposées :

1. **Filtre de seuil** (`shakeThreshold`) : si la g-force ne dépasse pas le seuil, la lecture est ignorée. Le seuil est ajusté en temps réel par le curseur de sensibilité, de 4,0 *g* (peu sensible) à 1,5 *g* (très sensible).
2. **Temps minimum entre secousses** (`SHAKE_SLOP_TIME_MS` = 500 ms) : empêche qu'un seul mouvement brusque soit compté comme plusieurs secousses tant que la lecture dépasse encore le seuil.
3. **Fenêtre temporelle de réinitialisation** (`SHAKE_COUNT_RESET_TIME_MS` = 3000 ms) : si plus de 3 secondes s'écoulent entre deux secousses valides, le compteur est remis à zéro. Cela impose que les secousses soient consécutives.
4. **Compteur minimal** (`MIN_SHAKE_COUNT` = 2) : au moins 2 secousses valides et consécutives sont nécessaires pour déclencher `onShakeDetected()`, évitant ainsi les activations accidentelles par un unique choc.

5.3. Constantes et seuil configurable

Les paramètres temporels sont des constantes de classe ; `shakeThreshold` est un champ mutable modifiable en temps réel par le curseur :

```

1 private static final int SHAKE_SLOP_TIME_MS      = 500;
2 private static final int SHAKE_COUNT_RESET_TIME_MS = 3000;
3 private static final int MIN_SHAKE_COUNT         = 2;
4
5 private float shakeThreshold = 2.75f;

```

Listing 8: Constantes de l'algorithme de détection — MainActivity.java

La formule de correspondance entre la position du curseur et le seuil :

```

1 seekBarSensitivity.setOnSeekBarChangeListener(
2     new SeekBar.OnSeekBarChangeListener() {
3         @Override
4         public void onProgressChanged(SeekBar seekBar,
5                                     int progress,
6                                     boolean fromUser) {
7             shakeThreshold = 4.0f - (progress * 0.25f);
8             tvSensitivityValue.setText(progress + " / 10");

```



```

9         graphAccelerometer.setThreshold(shakeThreshold);
10    }
11    @Override public void onStartTrackingTouch(SeekBar seekBar) {}
12    @Override public void onStopTrackingTouch(SeekBar seekBar) {}
13 });

```

Listing 9: Ajustement dynamique du seuil depuis le SeekBar — MainActivity.java

La formule $4.0 - \text{progress} * 0.25$ fait correspondre la plage $[0, 10]$ du curseur à la plage de seuil $[4,0\text{ g}, 1,5\text{ g}]$: plus le niveau de sensibilité est élevé (curseur vers la droite), plus le seuil est bas et plus il est facile de déclencher la détection. La valeur par défaut est $\text{progress} = 5$ ($\rightarrow 2,75\text{ g}$).

6. Graphique en Temps Réel : AccelerometerGraphView

6.1. Vue personnalisée avec Canvas

Pour visualiser les valeurs de l'accéléromètre en temps réel, la classe `AccelerometerGraphView` étend `android.view.View` et dessine directement sur un `Canvas`. Le graphique affiche :

- **Ligne bleue** : magnitude de la g-force en temps réel.
- **Ligne rouge pointillée** : seuil actuel, se déplace verticalement lorsque le curseur est déplacé.
- **Grille grise** : lignes horizontales tous les 1 g.

6.2. Tampon circulaire

La vue maintient un tampon circulaire de 150 échantillons. À chaque nouvelle lecture, la position la plus ancienne est écrasée et le pointeur `head` avance en modulo :

```

1 private static final int    BUFFER_SIZE = 150;
2 private static final float  Y_MAX      = 5.0f;
3
4 private final float[] values = new float[BUFFER_SIZE];
5 private int    head      = 0;
6 private int    count     = 0;
7 private float  threshold = 2.75f;
8
9 public void addValue(float gForce) {
10     values[head] = gForce;
11     head = (head + 1) % BUFFER_SIZE;
12     if (count < BUFFER_SIZE) count++;
13     postInvalidate();
14 }
15
16 public void setThreshold(float threshold) {
17     this.threshold = threshold;
18     postInvalidate();
19 }

```

Listing 10: Tampon circulaire et mise à jour — AccelerometerGraphView.java

`postInvalidate()` est *thread-safe* : il peut être appelé depuis n'importe quel thread et planifie un redessin sur le thread UI. L'axe Y du graphique couvre de 0 à 5 g (Y_MAX), plage qui contient en permanence la ligne de seuil (entre 1,5 g et 4,0 g).

6.3. Initialisation des pinceaux

Les objets `Paint` sont créés *uniquement dans le constructeur*, jamais dans `onDraw()`. C'est une règle fondamentale sous Android : instancier des objets dans `onDraw()` génère du garbage à chaque frame et dégrade significativement les performances.

```

1 public AccelerometerGraphView(Context context, AttributeSet attrs)
2 {
3     super(context, attrs);
4
5     linePaint = new Paint(Paint.ANTI_ALIAS_FLAG);
6     linePaint.setColor(0xFF42A5F5);           // bleu Material Blue 400
7     linePaint.setStrokeWidth(3f);
8     linePaint.setStyle(Paint.Style.STROKE);
9
10    thresholdPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
11    thresholdPaint.setColor(0xFFEF5350);       // rouge Material Red 400
12    thresholdPaint.setStrokeWidth(2f);
13    thresholdPaint.setStyle(Paint.Style.STROKE);
14    thresholdPaint.setPathEffect(
15        new DashPathEffect(new float[]{16f, 8f}, 0));
16
17    gridPaint = new Paint();
18    gridPaint.setColor(0x33FFFFFF);           // blanc a 20% d'opacite
19    gridPaint.setStrokeWidth(1f);
20 }
```

Listing 11: Initialisation des Paint dans le constructeur — AccelerometerGraphView.java

Le `DashPathEffect` de la ligne de seuil reçoit un tableau de deux valeurs : [16, 8] signifie segment de 16 px allumé suivi de 8 px éteint, produisant l'effet de ligne pointillée.

6.4. Méthode onDraw

`onDraw()` est exécutée à chaque redessin de la vue. Elle trace trois couches dans l'ordre sur le Canvas :

```

1 @Override
2 protected void onDraw(Canvas canvas) {
3     int w = getWidth();
4     int h = getHeight();
5
6     canvas.drawColor(0xFF1E1E1E);           // fond sombre VSCode
7
8     for (int g = 1; g <= 4; g++) {           // grille tous les 1g
9         float y = h * (1f - g / Y_MAX);
10        canvas.drawLine(0, y, w, y, gridPaint);
11    }
12
13    float ty = h * (1f - threshold / Y_MAX); // ligne de seuil
```

```

14     canvas.drawLine(0, ty, w, ty, thresholdPaint);
15
16     if (count < 2) return;
17
18     path.reset();
19     float xStep = (float) w / (BUFFER_SIZE - 1);
20     for (int i = 0; i < count; i++) {
21         int idx = (head - count + i + BUFFER_SIZE) % BUFFER_SIZE
22     ;
23         float x    = i * xStep;
24         float y    = h * (1f - Math.min(values[idx], Y_MAX) / Y_MAX
25     );
26         if (i == 0) path.moveTo(x, y);
27         else       path.lineTo(x, y);
28     }
29     canvas.drawPath(path, linePaint);
30 }

```

Listing 12: Tracé du graphique avec Canvas — AccelerometerGraphView.java

La coordonnée verticale en pixels pour une valeur v se calcule comme :

$$y_{\text{px}} = h \cdot \left(1 - \frac{v}{Y_{\text{max}}}\right)$$

Le facteur $(1 - \cdot)$ inverse l'axe car dans le système de coordonnées du Canvas Android l'axe Y croît vers le bas (l'origine est en haut à gauche). L'indice `idx` parcourt le tampon circulaire en ordre chronologique : de la donnée la plus ancienne (à gauche) à la plus récente (à droite), produisant l'effet de défilement vers la gauche au fur et à mesure que de nouveaux échantillons arrivent.

7. Exécution en Arrière-Plan : TorchService

7.1. Motivation

Le comportement par défaut d'Android est de suspendre l'accéléromètre lorsque l'Activité entre en pause (quand l'utilisateur quitte l'application). Pour maintenir la fonctionnalité active en arrière-plan, Android impose l'utilisation d'un **Foreground Service** : un service qui affiche une notification persistente, informant l'utilisateur que l'application est toujours active. Cela empêche le système d'exploitation de tuer le processus par manque de ressources.

7.2. Architecture du service

TorchService étend `Service` et implémente `SensorEventListener`, répliquant exactement la même logique de détection de secousse que `MainActivity` :

```

1  @Override
2  public int onStartCommand(Intent intent, int flags, int startId) {
3      if (intent != null) {

```

```

4      shakeThreshold = intent.getFloatExtra(EXTRA_THRESHOLD,
5      2.75f);
6      torchOn      = intent.getBooleanExtra(EXTRA_TORCH_STATE,
7      false);
8      }
9      createNotificationChannel();
10     startForeground(NOTIF_ID, buildNotification());
11     if (accelerometer != null) {
12         sensorManager.registerListener(
13             this,
14             accelerometer,
15             SensorManager.SENSOR_DELAY_NORMAL);
16     }
17     return START_STICKY;
18 }

```

Listing 13: Démarrage du service — TorchService.java

Le service reçoit le seuil de sensibilité et l'état courant du flash via les `Intent` extras, afin de reprendre là où l'Activité s'est arrêtée. `startForeground()` doit être appelé dans les 5 secondes suivant le démarrage du service. `START_STICKY` indique au système que, s'il doit terminer le processus, il doit le redémarrer automatiquement.

La notification persistente est construite avec `NotificationCompat` :

```

1  private Notification buildNotification() {
2      Intent intent = new Intent(this, MainActivity.class);
3      PendingIntent pi = PendingIntent.getActivity(
4          this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
5      return new NotificationCompat.Builder(this, CHANNEL_ID)
6          .setContentTitle("Helios actif")
7          .setContentText("Secouez pour allumer/eteindre la
8          torche")
9          .setSmallIcon(R.mipmap.ic_launcher)
10         .setContentIntent(pi)
11         .build();
12 }

```

Listing 14: Construction de la notification — TorchService.java

Le `PendingIntent` avec `FLAG_IMMUTABLE` permet à l'utilisateur de toucher la notification pour revenir à l'Activité.

7.3. Transfert de contrôle entre Activité et Service

La conception garantit qu'à tout moment *un seul* des deux composants écoute l'accéléromètre :

État de l'application	Interrupteur	Qui écoute
Au premier plan	Désactivé	MainActivity
Au premier plan	Activé	MainActivity
En arrière-plan	Désactivé	Personne (capteur inactif)
En arrière-plan	Activé	TorchService

Lorsque l'utilisateur revient dans l'application, `onResume()` arrête le service *avant* d'enregistrer le listener dans l'Activité :

```

1  @Override
2  protected void onResume() {
3      super.onResume();
4      stopService(new Intent(this, TorchService.class));
5      if (accelerometer != null) {
6          sensorManager.registerListener(
7              this,
8              accelerometer,
9              SensorManager.SENSOR_DELAY_NORMAL);
10     }
11 }

```

Listing 15: Reprise du contrôle par l'Activité — MainActivity.java

Lorsque l'utilisateur quitte l'application, onPause() désenregistre le listener de l'Activité et, si l'interrupteur est activé, lance le service :

```

1  @Override
2  protected void onPause() {
3      super.onPause();
4      if (accelerometer != null) {
5          sensorManager.unregisterListener(this);
6      }
7      if (backgroundEnabled) {
8          Intent intent = new Intent(this, TorchService.class);
9          intent.putExtra(TorchService.EXTRA_THRESHOLD,
10             shakeThreshold);
11          intent.putExtra(TorchService.EXTRA_TORCH_STATE, torchOn);
12          startForegroundService(intent);
13     }
14 }

```

Listing 16: Transfert vers le service lors de la mise en pause — MainActivity.java

7.4. Interrupteur dans l'interface

L'utilisateur contrôle le mode arrière-plan via un SwitchMaterial situé dans le coin supérieur droit de l'écran :

```

1  SwitchMaterial switchBackground = findViewById(R.id.
2      switchBackground);
3  switchBackground.setOnCheckedChangeListener(
4      (buttonView, isChecked) -> backgroundEnabled = isChecked);

```

Listing 17: Interrupteur du mode arrière-plan — MainActivity.java

7.5. Déclaration dans le Manifest

Pour qu'Android reconnaisse le service et lui permette de s'exécuter au premier plan, il est déclaré dans AndroidManifest.xml :

```

1  <uses-permission android:name="android.permission.
2      FOREGROUND_SERVICE"/>
3  <uses-permission android:name=
4      "android.permission.FOREGROUND_SERVICE_SPECIAL_USE"/>

```

```

4  <uses-permission android:name="android.permission.
    POST_NOTIFICATIONS"/>
5
6  <service
7      android:name=".TorchService"
8      android:foregroundServiceType="specialUse"
9      android:exported="false" />

```

Listing 18: Permissions et déclaration du service — `AndroidManifest.xml`

Le type `specialUse` est le plus adapté : la combinaison accéléromètre + contrôle du flash ne correspond à aucune des catégories spécifiques proposées par Android 14 (caméra, localisation, santé, etc.). `POST_NOTIFICATIONS` est requis pour afficher la notification persistente sous Android 13 et supérieur.

8. Conclusions

Android Helios intègre les deux modules matériels requis—accéléromètre et caméra/flash—de manière fonctionnelle et robuste. Les points clés de la conception sont :

- **Détection de secousse en quatre couches** : seuil de g-force + temps minimum entre secousses + fenêtre de réinitialisation + compteur minimal. Chaque couche réduit indépendamment les faux positifs.
- **TorchCallback comme source unique de vérité** : l'état de la lampe de poche reflète toujours l'état réel du matériel, indépendamment du composant qui l'a modifié (Activité, Service ou autre application).
- **Graphique comme outil de calibrage** : l'utilisateur peut voir en temps réel la magnitude de ses mouvements et ajuster le seuil jusqu'à ce que la ligne rouge se trouve en dessous de ses secousses et au-dessus du bruit de fond.
- **Cycle de vie strict** : le capteur ne fonctionne jamais en arrière-plan sans le consentement explicite de l'utilisateur, et lorsqu'il le fait, le système d'exploitation est correctement notifié via le Foreground Service avec sa notification persistente.